

LINKED LIST

```
#include <stdio.h>

#include <stdlib.h>

struct node{
    int data;
    struct node* next;
};

void insertAtBeginning(int val, struct node** head){
    struct node* newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = val;
    newnode->next = (*head) ;
    *head = newnode;
}

void insertAfter(int val, struct node* prevnode){
    if(prevnode == NULL){
        printf("given previous node cant be null");
        return ;
    }
    struct node* newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = val;
    newnode->next = prevnode->next;
    prevnode->next = newnode;
}
```

```

void insertAtEnd(int val, struct node** head){
    struct node* temp = (*head);
    struct node* newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = val;
    newnode->next = NULL;
    if (*head == NULL) {
        *head = newnode;
        return;
    }
    while(temp->next != NULL){
        temp = temp->next;
    }
    temp->next = newnode;
    return;
}

```

```

void deletenode(struct node** head, int val){
    if(head == NULL){
        printf("head cant be null");
        return ;
    }else{
        struct node* prev = NULL;
        struct node* temp = *head;
        if (temp->data == val) {
            *head = temp->next;

```

```

        free(temp);
        return;
    }
    while(temp->next != NULL && temp->data == val){
        prev = prev->next;
        temp = temp->next;
    }
    if(temp->next == NULL){
        printf("error, value not found");
        return;
    }
    prev->next = temp->next;
    return ;
}

}

int searchnode(struct node** head, int val){
    struct node* temp = *head;
    while(temp->data != val && temp->next != NULL)
        temp = temp->next;
    if(temp->data == val)
        return 1;
    if(temp->next == NULL)
        return 0;
}

void sort(struct node** head){

```

```

struct node* index = (*head)->next;
struct node* current = (*head);
int temp;
if(*head == NULL)
    return;
else{
    while(current != NULL){
        index = current->next;
        while (index != NULL) {
            if (current->data > index->data) {
                temp = current->data;
                current->data = index->data;
                index->data = temp;
            }
            index = index->next;
        }
        current = current->next;
    }
}

void printList(struct node* Node) {
    while (Node != NULL) {
        printf(" %d ", Node->data);
        Node = Node->next;
    }
}

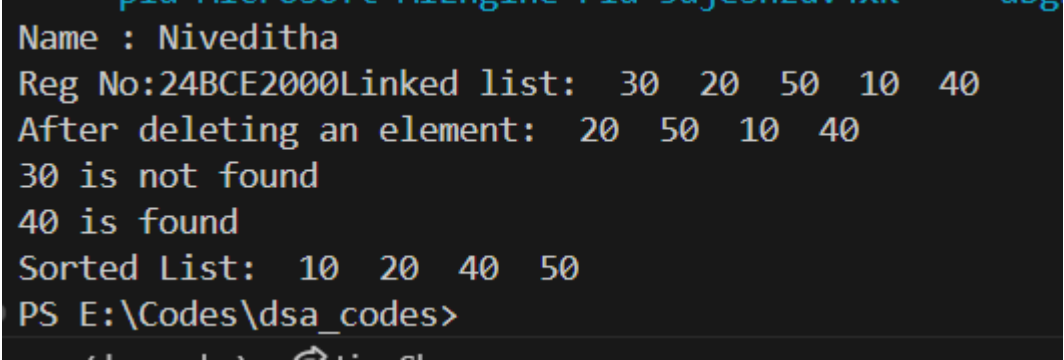
```

```

int main()
{
    printf("Name : Niveditha \nReg No:24BCE2000");
    struct node* head = NULL;
    insertAtEnd(10, &head);
    insertAtBeginning(20, &head);
    insertAtBeginning(30, &head);
    insertAtEnd(40, &head);
    insertAfter(50, head->next);
    printf("Linked list: ");
    printList(head);
    printf("\nAfter deleting an element: ");
    deletenode(&head, 30);
    printList(head);
    int item_to_find = 30;
    if (searchnode(&head, item_to_find))
        printf("\n%d is found", item_to_find);
    else
        printf("\n%d is not found", item_to_find);
    item_to_find = 40;
    if (searchnode(&head, item_to_find))
        printf("\n%d is found", item_to_find);
    else
        printf("\n%d is not found", item_to_find);
    sort(&head);
    printf("\nSorted List: ");

```

```
printList(head);  
}
```



```
Name : Niveditha  
Reg No:24BCE2000Linked list: 30 20 50 10 40  
After deleting an element: 20 50 10 40  
30 is not found  
40 is found  
Sorted List: 10 20 40 50  
PS E:\Codes\dsa_codes>
```

DOUBLE LINKED LIST

```
#include <stdio.h>  
  
#include <stdlib.h>  
  
struct node{  
    int data;  
    struct node* next;  
    struct node* prev;  
};  
  
void insertAtBeginning(int val, struct node** head){  
    struct node* newnode = (struct node*)malloc(sizeof(struct node));  
    newnode->data = val;  
    newnode->next = (*head) ;  
    newnode->prev = NULL;  
    if(*head != NULL)  
        (*head)->prev = newnode;  
    *head = newnode;  
}
```

```

void insertAfter(int val, struct node* prevnode){
    if(prevnode == NULL){
        printf("given previous node cant be null");
        return ;
    }
    struct node* newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = val;
    newnode->next = prevnode->next;
    newnode->prev = prevnode;
    prevnode->next = newnode;
    if(prevnode->next != NULL)
        prevnode->next->prev = newnode;
}

```

```

void insertAtEnd(int val, struct node** head){
    struct node* temp = (*head);
    struct node* newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = val;
    newnode->next = NULL;
    if (*head == NULL) {
        newnode->prev = NULL;
        *head = newnode;
        return;
    }
    while(temp->next != NULL){

```

```

        temp = temp->next;
    }
    temp->next = newnode;
    newnode->prev = temp;
    return;
}

void deletenode(struct node** head, int val){
    if(*head == NULL){
        printf("head cant be null");
        return;
    }
    struct node* temp = *head;
    while(temp != NULL && temp->data != val)
        temp = temp->next;
    if(temp == NULL){
        printf("error, value not found");
        return;
    }
    if(temp->prev != NULL)
        temp->prev->next = temp->next;
    else
        *head = temp->next;
    if(temp->next != NULL)
        temp->next->prev = temp->prev;
    free(temp);
}

```



```

int searchnode(struct node** head, int val){
    struct node* temp = *head;
    while(temp->data != val && temp->next != NULL)
        temp = temp->next;
    if(temp->data == val)
        return 1;
    if(temp->next == NULL)
        return 0;
}

void printforward(struct node* Node) {
    while (Node != NULL) {
        printf(" %d ", Node->data);
        Node = Node->next;
    }
}

void printbackward(struct node* Node){
    struct node* last = Node;
    if(Node == NULL)
        return;
    while(last->next != NULL)
        last = last->next;
    while(last != NULL){
        printf(" %d ", last->data);
        last = last->prev;
    }
}

```

```

}

int main()
{
    printf("Name : Niveditha \nReg No:24BCE2000");
    struct node* head = NULL;
    insertAtEnd(10, &head);
    insertAtBeginning(20, &head);
    insertAtBeginning(30, &head);
    insertAtEnd(40, &head);
    insertAfter(50, head->next);
    printf("Linked list: ");
    printforward(head);
    printf("\nAfter deleting an element: ");
    deletenode(&head, 30);
    printforward(head);
    int item_to_find = 30;
    if (searchnode(&head, item_to_find))
        printf("\n%d is found", item_to_find);
    else
        printf("\n%d is not found", item_to_find);
    item_to_find = 40;
    if (searchnode(&head, item_to_find))
        printf("\n%d is found", item_to_find);
    else
        printf("\n%d is not found", item_to_find);
}

```

}

```
Name : Niveditha
Reg No:24BCE2000Linked list: 30 20 50 10 40
After deleting an element: 20 50 10 40
30 is not found
40 is found
PS E:\Codes\dsa_codes>
```

STACKS USING LINKED LIST

```
#include <stdio.h>

#include <stdlib.h>

struct node{
    int data;
    struct node* next;
};

struct node* head = NULL;

void push(int val){
    struct node* newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = val;
    newnode->next = head; //make the newnode point to the current top
    element
    head = newnode; //change the head to top element
}

int pop(){
    if(head == NULL){
        printf("\nStack empty\n");
    }
}
```

```

        return -1;
    }
    else{
        struct node* temp = head;
        int val = head->data;
        head = head->next; //u r basically going down one level in the stack.
        free(temp);
        return val;
    }
}

void print(){
    struct node* temp = head;
    printf("elements in the stack are = ");
    while(temp != NULL){
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main(){
    printf("Name : Niveditha \nReg No:24BCE2000");
    push(15);
    push(35);
    push(90);
    print();
    printf("element popped = %d\n",pop());
}

```

```
print();  
}
```

```
Name : Niveditha  
Reg No:24BCE2000elements in the stack are = 90 35 15 NULL  
element popped = 90  
elements in the stack are = 35 15 NULL  
PS E:\Codes\dsa_codes>
```

QUEUES USING LINKED LIST

```
#include <limits.h>  
  
#include <stdio.h>  
  
#include <stdlib.h>  
  
typedef struct Node {  
    int data;  
    struct Node* next;  
} Node;  
  
Node* createNode(int new_data) {  
    Node* new_node = (Node*)malloc(sizeof(Node));  
    new_node->data = new_data;  
    new_node->next = NULL;  
    return new_node;  
}  
  
typedef struct Queue {  
    Node *front, *rear;  
} Queue;
```

```

Queue* createQueue() {
    Queue* q = (Queue*)malloc(sizeof(Queue));
    q->front = q->rear = NULL;
    return q;
}

int isEmpty(Queue* q) {
    if (q->front == NULL && q->rear == NULL) {
        return 1;
    }
    return 0;
}

void enqueue(Queue* q, int new_data) {
    Node* new_node = createNode(new_data);
    if (q->rear == NULL) {
        q->front = q->rear = new_node;
        return;
    }
    q->rear->next = new_node;
    q->rear = new_node;
}

void dequeue(Queue* q) {
    if (isEmpty(q)) {
        printf("Queue Underflow\n");
        return;
    }
    Node* temp = q->front;

```

```

    q->front = q->front->next;
    if (q->front == NULL)
        q->rear = NULL;
    free(temp);
}

int getFront(Queue* q) {
    if (isEmpty(q)) {
        printf("Queue is empty\n");
        return INT_MIN;
    }
    return q->front->data;
}

int getRear(Queue* q) {
    if (isEmpty(q)) {
        printf("Queue is empty\n");
        return INT_MIN;
    }
    return q->rear->data;
}

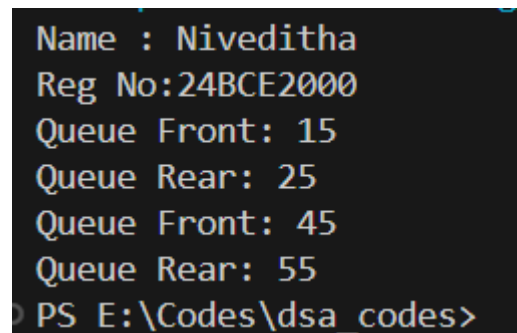
int main()
{
    printf("Name : Niveditha \nReg No:24BCE2000\n");
    Queue* q = createQueue();
    enqueue(q, 15);
    enqueue(q, 25);
    printf("Queue Front: %d\n", getFront(q));
}

```

```

printf("Queue Rear: %d\n", getRear(q));
dequeue(q);
dequeue(q);
enqueue(q, 35);
enqueue(q, 45);
enqueue(q, 55);
dequeue(q);
printf("Queue Front: %d\n", getFront(q));
printf("Queue Rear: %d\n", getRear(q));
return 0;
}

```



```

Name : Niveditha
Reg No:24BCE2000
Queue Front: 15
Queue Rear: 25
Queue Front: 45
Queue Rear: 55
PS E:\Codes\dsa_codes>

```

POLYNOMIAL ADDITION

```

#include <stdio.h>
#include <stdlib.h>
struct Node {
    int coeff;
    int pow;
    struct Node* next;
};
struct Node* createNode(int c, int p);

```



```

struct Node* addPolynomial(struct Node* head1, struct Node* head2) {
    struct Node* dummy = createNode(0, 0);
    struct Node* prev = dummy;
    struct Node *curr1 = head1, *curr2 = head2;
    while (curr1 != NULL && curr2 != NULL) {
        if (curr1->pow < curr2->pow) {
            prev->next = curr2;
            prev = curr2;
            curr2 = curr2->next;
        }
        else if (curr1->pow > curr2->pow) {
            prev->next = curr1;
            prev = curr1;
            curr1 = curr1->next;
        }
        else {
            curr1->coeff = curr1->coeff + curr2->coeff;
            prev->next = curr1;
            prev = curr1;
            curr1 = curr1->next;
            curr2 = curr2->next;
        }
    }
    if (curr1 != NULL) {
        prev->next = curr1;
    }
}

```

```

    if (curr2 != NULL) {
        prev->next = curr2;
    }
    return dummy->next;
}

void printList(struct Node* head) {
    struct Node* curr = head;
    while (curr != NULL) {
        printf("%d,%d ", curr->coeff, curr->pow);
        curr = curr->next;
    }
    printf("\n");
}

struct Node* createNode(int c, int p) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->coeff = c;
    newNode->pow = p;
    newNode->next = NULL;
    return newNode;
}

int main() {
    printf("Name : Niveditha \nReg No:24BCE2000\n");
    // 1st polynomial: 5x^2+4x^1+2x^0
    struct Node* head1 = createNode(5, 2);
    head1->next = createNode(4, 1);
    head1->next->next = createNode(2, 0);
}

```

```
// 2nd polynomial: -5x^1-5x^0
struct Node* head2 = createNode(-5, 1);
head2->next = createNode(-5, 0);
struct Node* head = addPolynomial(head1, head2);
printList(head);
return 0;
}
```

```
Name : Niveditha
Reg No:24BCE2000
5,2 -1,1 -3,0
PS E:\Codes\dsa_codes>
```